

Newsworthy A2A Protocol

Newsworthy implements the [Agent-to-Agent \(A2A\) protocol](#), allowing AI agents to discover and interact with published press releases over a standard JSON-RPC 2.0 interface.

Discovery

Agents discover Newsworthy by fetching the Agent Card:

```
GET /.well-known/agent-card.json
```

The Agent Card returns the endpoint URL, supported capabilities, and available skills.

Endpoint

All A2A requests are sent as JSON-RPC 2.0 POST requests to:

```
POST /api/a2a
```

Content-Type: `application/json`

Public skills require no authentication. Authenticated skills require a Bearer token (API key).

Authentication

Authenticated access uses per-user/brand API keys passed as Bearer tokens.

Creating API Keys

1. Visit `/settings/api-keys` in the dashboard
2. Click "Create API Key" and select a brand
3. Copy the key immediately — it is shown only once

Using API Keys

Pass the key as a Bearer token in the `Authorization` header:

```
Authorization: Bearer nw_a2a_abc123...
```

Each API key is scoped to a **user + brand pair**. Authenticated skills will operate on the brand associated with the key.

Key Format

Keys follow the format `nw_a2a_{32 random hex chars}`. Only a bcrypt hash is stored server-side.

Key Management API

- `GET /api/a2a/keys` — List your API keys (requires session auth)
- `POST /api/a2a/keys` — Create a new key: `{ name, companyUuid }`
- `DELETE /api/a2a/keys?uuid=...` — Revoke a key

Rate Limiting

- **60 requests per minute** per IP address (unauthenticated)
- **120 requests per minute** per API key (authenticated)
- Rate limit headers are included on every response:
 - `X-RateLimit-Remaining` — requests left in the current window
 - `X-RateLimit-Reset` — timestamp when the window resets
- Exceeding the limit returns HTTP 429 with a `Retry-After` header

Methods

message/send

Send a message and receive a completed task synchronously.

```
{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{ "type": "text", "text": "search for technology press releases" }]
    },
    "metadata": {
      "skillId": "search_releases"
    }
  },
  "id": "req-1"
}
```

Response: A JSON-RPC response containing a `Task` object with status, messages, and artifacts.

message/stream

Same as `message/send` but returns an SSE (Server-Sent Events) stream with real-time status updates.

```
{
  "jsonrpc": "2.0",
  "method": "message/stream",
  "params": {
    "message": {
      "role": "user",
      "parts": [{ "type": "text", "text": "search for technology press releases" }]
    }
  },
  "id": "req-2"
}
```

Response: `text/event-stream` with events:

```
data: {"type":"TaskStatusUpdate","taskId":"...","status":
{"state":"submitted","timestamp":"..."}}
```

```
data: {"type":"TaskStatusUpdate","taskId":"...","status":
{"state":"working","timestamp":"..."}}
```

```
data: {"type":"TaskArtifactUpdate","taskId":"...","artifact":{"..."}}
```

```
data: {"type":"TaskStatusUpdate","taskId":"...","status":
{"state":"completed","timestamp":"..."}}
```

tasks/get

Retrieve the current state of a task by ID.

```
{
  "jsonrpc": "2.0",
  "method": "tasks/get",
  "params": { "taskId": "task-uuid-here" },
  "id": "req-3"
}
```

tasks/cancel

Cancel a running task.

```
{
  "jsonrpc": "2.0",
  "method": "tasks/cancel",
  "params": { "taskId": "task-uuid-here" },
  "id": "req-4"
}
```

Skills

search_releases

Search published press releases by keyword, category, region, or date range.

Text input — the message text is used as the search query:

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "find press releases about AI" }]
  }
}
```

Structured input — pass a JSON data part for advanced filtering:

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": {
        "query": "artificial intelligence",
        "categoryId": 5,
        "regionId": 12,
        "dateFrom": "2025-01-01",
        "dateTo": "2025-12-31",
        "limit": 10,
        "offset": 0
      }
    }]
  }
}
```

Parameter	Type	Description
<code>query</code>	string	Text search across title, abstract, body
<code>categoryId</code>	number	Filter by category ID
<code>regionId</code>	number	Filter by region ID
<code>dateFrom</code>	string	ISO date — releases published on or after
<code>dateTo</code>	string	ISO date — releases published on or before
<code>limit</code>	number	Results per page (default 20, max 50)
<code>offset</code>	number	Pagination offset (default 0)

Returns: Array of releases with uuid, title, abstract, location, companyName, releasedAt, categories, and regions.

search_brands

Search brands (companies) that have published press releases, along with their 5 most recent releases. Only brands with at least one published release are returned. No contact information is exposed.

Text input — the message text is used as the search query:

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "find brands in healthcare" }]
  }
}
```

Structured input — pass a JSON data part:

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": {
        "query": "healthcare",
        "limit": 10,
        "offset": 0
      }
    }]
  }
}
```

Parameter	Type	Description
query	string	Text search across name, description, city, state
limit	number	Results per page (default 20, max 50)
offset	number	Pagination offset (default 0)

Returns per brand:

Field	Type	Description
uuid	string	Brand UUID
name	string	Company name
description	string	Newsroom description
newsroomUri	string	Newsroom slug
logoUrl	string	Logo image URL
website	string	Company website
location	string	City, state, country
publishedReleaseCount	number	Total published releases
recentReleases	array	Up to 5 latest releases (uuid, title, abstract, releasedAt, slug)

get_release

Retrieve the full content of a press release by UUID or slug.

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "get release a1b2c3d4-e5f6-7890-abcd-ef1234567890" }]
  }
}
```

A UUID anywhere in the message text will be automatically extracted. You can also pass it via a data part:

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": { "uuid": "a1b2c3d4-e5f6-7890-abcd-ef1234567890" }
    }]
  }
}
```

Returns: Full release content including title, abstract, body, pullquote, location, company info (name, website, logo, newsroom URI), categories, regions, FAQs, images, and readability scores. No contact information is included.

analyze_release

Analyze a published press release for SEO quality, readability, and key entities. Uses AI analysis with automatic caching.

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "analyze a1b2c3d4-e5f6-7890-abcd-ef1234567890"
    }]
  }
}
```

Returns:

Field	Type	Description
seoScore	number	SEO optimization score (0-100)
readabilityScore	number	Overall readability score (0-100)
fleschEase	number	Flesch reading ease (from release)
readTime	number	Estimated read time in minutes
keyEntities	string[]	People, orgs, products mentioned
summary	string	One-paragraph summary
suggestedImprovements	string[]	Specific improvement recommendations

Authenticated Skills

These skills require a Bearer token. Pass the API key in the `Authorization` header.

create_brand

Create a new brand/company for the authenticated user.

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": {
        "companyName": "Acme Corp",
        "website": "https://acme.com",
        "city": "San Francisco",
        "state": "CA",
        "countryCode": "US"
      }
    }]
  }
}
```

Parameter	Type	Required	Description
companyName	string	Yes	Brand/company name
website	string	No	Company website URL
city	string	No	City
state	string	No	State (2-letter)
countryCode	string	No	Country code

Returns: { uuid, name, newsroomUri }

update_brand

Update the brand associated with the API key's company.

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": { "companyName": "Acme Corp International", "website":
"https://acme.global" }
    }]
  }
}
```

Updatable fields: companyName, website, city, state, countryCode, addr1, addr2, postalCode, phone, email, linkedinUrl, xUrl, youtubeUrl, instagramUrl.

list_brands

List all brands the authenticated user has access to (owned + team memberships).

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "list my brands" }]
  }
}
```

Returns: Array of { uuid, name, role, newsroomUri, publishedReleaseCount }

create_release

Create a new press release draft. Consumes 1 credit from the API key's brand or user-level credits.

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": {
        "title": "Acme Corp Launches New Product",
        "abstract": "Acme Corp announces...",
        "body": "<p>Full release content here...</p>",
        "location": "San Francisco, CA",
        "categoryIds": [5, 12],
        "regionIds": [1]
      }
    }]
  }
}
```

Parameter	Type	Required	Description
title	string	No	Release title (max 180 chars)
abstract	string	No	Short summary
body	string	No	Full HTML body
pullquote	string	No	Pull quote
location	string	No	Dateline location
categoryIds	number[]	No	Category IDs
regionIds	number[]	No	Region IDs

Returns: { uuid, title, status: "draftnxt" }

update_release

Update a draft release. Blocked if status is review , approved , or published .

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": {
        "uuid": "abc123...",
        "title": "Updated Title",
        "body": "<p>Updated content</p>"
      }
    ]
  }
}
```

Parameter	Type	Required	Description
uuid	string	Yes	Release UUID

All other fields from `create_release` can be updated.

delete_release

Soft-delete a release and reallocate credits. Blocked if status is `approved`, `sent`, or `review`.

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "delete release abc123..." }]
  }
}
```

Returns: `{ success: true }`

submit_release

Submit a release for editorial review. Changes status from `draftnxt / draft` to `review`.

```
{
  "message": {
    "role": "user",
    "parts": [{ "type": "text", "text": "submit release abc123..." }]
  }
}
```

Returns: `{ uuid, status: "review" }`

list_releases

List releases for the API key's brand. Supports optional status filter.

```
{
  "message": {
    "role": "user",
    "parts": [{
      "type": "data",
      "mimeType": "application/json",
      "data": { "status": "draftnxt", "limit": 10, "offset": 0 }
    }]
  }
}
```

Parameter	Type	Description
status	string	Filter by status (draftnxt, review, sent...)
limit	number	Results per page (default 20, max 50)
offset	number	Pagination offset (default 0)

Returns: Array of { uuid, title, status, createdAt, releasedAt }

Skill Routing

You can specify which skill to invoke in two ways:

1. **Explicit** — set `metadata.skillId` in the request params:

```
{ "metadata": { "skillId": "search_releases" } }
```

2. **Inferred** — the agent infers the skill from message text:

Public skills (no auth):

- Words like *brand, brands, company, companies, newsroom* route to `search_brands`
- Words like *search, find, list, query, browse* route to `search_releases`
- Words like *analyze, review, assess, evaluate* route to `analyze_release`
- Words like *get, read, fetch, show* or a UUID pattern route to `get_release`

- If nothing matches, defaults to `search_releases`

Authenticated skills (with Bearer token):

- *create brand/company* routes to `create_brand`
- *update brand/company* routes to `update_brand`
- *list brands/my brands* routes to `list_brands`
- *create release/pr* routes to `create_release`
- *update release/pr* routes to `update_release`
- *delete release/pr* routes to `delete_release`
- *submit release/pr* routes to `submit_release`
- *list releases/my releases* routes to `list_releases`
- If no authenticated skill matches, falls through to public skills

Task Lifecycle

Every request creates a Task that progresses through states:

```
submitted → working → completed
                → failed
                → canceled
```

Tasks are stored in memory and automatically cleaned up after 1 hour.

Error Codes

Standard JSON-RPC 2.0 error codes:

Code	Meaning
-32700	Parse error
-32600	Invalid request
-32601	Method not found
-32602	Invalid params
-32603	Internal error
-32001	Task not found
401	Invalid/expired API key
429	Rate limit exceeded

A2A Client

Newsworthy includes a TypeScript client for consuming other A2A agents:

```
import { A2AClient } from '@lib/a2a/client'

const client = new A2AClient({ timeout: 30_000 })

// Discover an agent
const card = await client.discoverAgent('https://other-agent.example.com')

// Send a message
const task = await client.sendMessage(card.url, {
  role: 'user',
  parts: [{ type: 'text', text: 'hello' }],
})

// Stream a message
for await (const event of client.streamMessage(card.url, message)) {
  console.log(event.type, event)
}

// Check task status
const status = await client.getTask(card.url, task.id)

// Cancel a task
await client.cancelTask(card.url, task.id)
```

curl Examples

Discover the agent:

```
curl https://newsworthy.ai/.well-known/agent-card.json
```

Search releases:

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "search for technology press releases"}]
    },
    "metadata": {"skillId": "search_releases"}
  },
  "id": "1"
}'
```

Search brands:

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "find healthcare brands"}]
    },
    "metadata": {"skillId": "search_brands"}
  },
  "id": "2"
}'
```

Get a release:

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "get release a1b2c3d4-e5f6-7890-abcd-ef1234567890"}]
    },
    "metadata": {"skillId": "get_release"}
  },
  "id": "3"
}'
```

Analyze a release:

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "analyze a1b2c3d4-e5f6-7890-abcd-ef1234567890"}]
    },
    "metadata": {"skillId": "analyze_release"}
  },
  "id": "4"
}'
```

Stream a search:

```
curl -N -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/stream",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "search for healthcare releases"}]
    }
  },
  "id": "5"
}'
```

Create a release (authenticated):

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-H "Authorization: Bearer nw_a2a_your_key_here" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{
        "type": "data",
        "mimeType": "application/json",
        "data": {
          "title": "My Press Release",
          "abstract": "A brief summary",
          "body": "<p>Full content</p>"
        }
      }]
    },
    "metadata": {"skillId": "create_release"}
  },
  "id": "6"
}'
```

List my releases (authenticated):

```
curl -X POST https://newsworthy.ai/api/a2a \
-H "Content-Type: application/json" \
-H "Authorization: Bearer nw_a2a_your_key_here" \
-d '{
  "jsonrpc": "2.0",
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "list my releases"}]
    },
    "metadata": {"skillId": "list_releases"}
  },
  "id": "7"
}'
```